

# Viva Preparation — Q&A Cheat Sheet

**Project:** Semantic Image Search Engine using CLIP for Cross-Modal Retrieval and Vision-Language Alignment

**Student:** Dhruv Singhal (A2305224486)

**Supervisor:** Dr. Rakesh Chandra Joshi

**Date:** July 2026

**Viva length:** ~15-20 minutes

**Hinglish strategy:** Answer in clear English with key Hinglish phrases when explaining complex ideas naturally. The professor is your supervisor, he's seen all your work, so keep it conversational and confident.

## Quick Self-Intro (30 sec, in case they ask)

"Good morning/afternoon sir. I'm Dhruv Singhal, B.Tech CSE 2024-28, enrollment A2305224486. My NTCC project is 'Semantic Image Search Engine using CLIP' under your guidance. It's a cross-modal retrieval system that lets users search the Flickr8k image gallery using text, images, or both — and benchmarks CLIP against BLIP-2 and ALIGN on the RF/telemetry domain. I built a 5-tab Streamlit app, deployed it on Streamlit Cloud, and integrated 3 datasets. The viva will cover the 11 sections on the outline slide."

## Section 1: Motivation & Problem

### Q1.1 — What is the "vocabulary mismatch" problem?

Lexical retrieval (BM25, TF-IDF) matches exact word tokens. So if a query says "a vivacious furry companion darting across a sandy waterfront" and the gold caption is "dog running on beach", they share 0 tokens — BM25 returns 0.0. Dense latent retrieval (CLIP) maps both into the same 256-d space where they end up close. This is the entire motivation for VLM-based retrieval.

### Q1.2 — Why not just fine-tune the whole ViT-H/14 on your data?

Three reasons:

1. **VRAM cost** — full fine-tuning of H/14 needs >40 GB (we only have 16 GB on Kaggle T4).
2. **Catastrophic forgetting** — overfitting on 8K pairs destroys the encoder's open-vocabulary zero-shot ability, which is exactly what makes CLIP useful in the first place.
3. **Overfitting** — 8K pairs is way too small to fine-tune 1B parameters without memorising noise.

So we do **PEFT** — freeze 99% of the backbone, train only the small cross-modal projection head.

### Q1.3 — What is the specific application you target?

RF (radio-frequency) telemetry and electromagnetic spectrograms. The use case is "search the spectrogram database using natural-language descriptions". We curate an 8K-pair subset of Flickr30k filtered by keywords like radio, frequency, QAM, modulation, OFDM, spectrum, etc.

## Section 2: Research Objectives & Contributions

### Q2.1 — What are your 5 research objectives?

1. Parameter-efficient domain adaptation of CLIP to RF/spectrogram domain
2. Cross-modal benchmarking (CLIP vs BLIP-2 vs ALIGN)
3. Empirical validation of dense > sparse retrieval on zero-overlap queries
4. ONNX + INT8 quantisation → <100 ms CPU latency
5. Analysis of alternative loss functions (InfoNCE vs SigLIP)

## Q2.2 — What's the single biggest contribution?

Demonstrating that 4 previously disjoint VLM subfields (PEFT, multi-modal prompt learning, quantisation, dense retrieval) can be **combined harmoniously** in one working system. Before this work, each subfield was studied in isolation.

## Q2.3 — How is this different from CLIP-Adapter?

CLIP-Adapter is a 1-layer linear-down / ReLU / linear-up bottleneck with a learnable residual ratio  $\alpha$ . Our 2-layer MLP (512→384→256, GELU, dropout 0.1) is a strictly more expressive version of the same idea. We measured +2.5 pp P@1 from this design choice alone (MaPLe thesis supports this).

---

# Section 3: Related Work

## Q3.1 — What is MaPLe and how does it relate to your work?

MaPLe (Multi-modal Prompt Learning, Khattak 2023) showed deeper branch-aware prompt structures capture richer cross-modal alignment than shallow linear probes. Our 2-layer MLP head draws direct inspiration — but instead of putting prompts in the transformer's input layers, we put non-linearity in the projection head. Functionally similar, architecturally simpler.

## Q3.2 — What is SigLIP and why didn't you use it?

SigLIP (Zhai 2023) replaces the symmetric InfoNCE softmax with a pairwise sigmoid loss. Each image-text pair is an independent binary classification (positive/negative). It scales better to large batches (2K+) because there's no softmax denominator coupling all examples.

I kept InfoNCE because:

- Our batch size is 128 — InfoNCE is stable and well-validated at this scale.
- SigLIP is documented as a future direction (Section VI.C.2 of thesis).
- I anticipate +2-4 pp gain at P@1 if we scale to batch=2K — that's the next iteration.

## Q3.3 — What's MobileCLIP and why is it relevant?

MobileCLIP (Apple, 2024) uses RLHF (caption rewriting + teacher-student distillation) to compress a large VLM into a tiny one for edge devices. Our ONNX + INT8 quantisation pipeline is the deployment-side analogue — we're not retraining a smaller model, but we are quantising an existing one to fit on edge hardware. The techniques (per-channel weight quantisation, dynamic activation quantisation) are directly from the Q-VLM and MobileCLIP literature.

---

# Section 4: Datasets

## Q4.1 — Why Flickr8k and not COCO?

- Flickr8k is small enough to fine-tune on CPU (8K pairs).
- Has high-quality natural-language captions (not just alt-text).
- Standard benchmark in retrieval literature.
- For the demo, we use a 300-image subset so the cloud app stays within the 1 GB sandbox memory limit.

## Q4.2 — How did you build the 8K RF/telemetry subset?

- Start with Flickr30k standard archive (~30K images with 5 captions each).
- Filter by caption keywords: radio, frequency, spectrum, signal, wave, modulation, transmission, antenna, wireless, QAM, PSK, FSK, OFDM, channel, interference.
- This gives ~9,200 candidate pairs.
- Uniformly sample 8,000 for the final dataset.
- Split: 6,400 train / 1,600 validation with deterministic seed.

## Q4.3 — Why split 80/20 and not 90/10 or 70/30?

- 80/20 is the literature standard for medium-sized retrieval corpora.

- 6,400 training pairs is enough for projection-head convergence in ~3 epochs.
  - 1,600 validation pairs gives statistically meaningful P@K numbers (margin of error ~2%).
- 

## Section 5: Methodology

### Q5.1 — Walk me through the architecture.

Dual-tower CLIP:

- **Image tower:** ViT-B/32 or ViT-L/14 (frozen) → 512-d feature
- **Text tower:** Transformer encoder (frozen) → 512-d feature
- **Projection head** (trainable): 2-layer MLP, 512→384→256, GELU + dropout 0.1 → 256-d embedding
- **L2 normalise** each embedding → unit vector on the 256-d hypersphere
- **Cosine similarity** = simple dot product
- **FAISS IndexFlatIP** for sub-millisecond exact top-K lookup

### Q5.2 — Why is the loss symmetric?

Two directions: image-to-text and text-to-image. Each is a softmax cross-entropy over the in-batch negatives, with  $\tau=0.07$ . We average the two. This ensures the embedding space has consistent geometry in both directions — critical because the retrieval engine must support queries in both directions using the same joint space.

### Q5.3 — What is warm-start initialisation and why does it matter?

At  $t=0$ , the new MLP's output is initialised to resemble the original CLIP embedding projected onto its first 256 principal components (under L2). This is an "identity-like" mapping within the original subspace. Training then smoothly reshapes the geometry without destabilising gradients. **Empirically: +9.7 pp at P@1 from PEFT alone** — the warm-start is what makes this work; random init would just give noise.

### Q5.4 — What is the InfoNCE loss formula?

For a batch of  $N$  (image, text) pairs, with embeddings  $z_i$  (image) and  $z_t$  (text), temperature  $\tau=0.07$ :

- Image-to-text loss:  $-1/N \times \sum_i \log( \exp(\text{sim}(z_i, z_{t,i}) / \tau) / \sum_j \exp(\text{sim}(z_i, z_{t,j}) / \tau) )$
- Text-to-image loss: symmetric
- Total =  $(I \rightarrow T + T \rightarrow I) / 2$

### Q5.5 — Why dropout 0.1?

Empirically validated — gave +1.2 pp at P@1 over no-dropout. With 6,400 training pairs and a 590K-parameter head, overfitting is a real risk, so dropout is necessary. 0.1 is the standard starting point that worked.

---

## Section 6: PEFT & Domain Adaptation

### Q6.1 — How many parameters exactly are trainable?

- 2-layer MLP,  $512 \times 384 + 384 \times 256 + 384$  (bias) + 256 (bias)  $\approx 295K$  params per tower
- 2 towers (image + text) → ~590K trainable parameters
- ViT-B/32 backbone: ~151M total → 0.40% trainable
- ViT-L/14 backbone: ~430M total → 0.14% trainable
- ViT-H/14 backbone: ~1.0B total → 0.06% trainable

### Q6.2 — Why these specific numbers (512→384→256)?

- 512 is CLIP's native embedding dimension.
- 384 is the standard "intermediate width" in transformers ( $\approx 0.75 \times \text{input}$ ).
- 256 is the FAISS index dimension — chosen so each embedding is exactly 256 floats (1 KB at fp32) for fast cache + retrieval.
- The 2-layer structure gives a non-linear mapping that captures richer cross-modal alignment (validated in the ablation: +2.5 pp over 1-layer linear).

### Q6.3 — Compare with CoOp, CLIP-Adapter, MaPLe.

- **CoOp**: learnable soft prompts in the text encoder. ~0.01% trainable. Less expressive than adapter methods.
  - **CLIP-Adapter**: 1-layer bottleneck + learnable residual  $\alpha$ . ~0.1% trainable.
  - **MaPLe**: deep branch-aware prompts at every transformer block. ~0.1% trainable. Most expressive but most complex.
  - **Ours (2-layer MLP head)**: ~0.1% trainable. Architecturally simpler than MaPLe (no transformer-internal forward pass changes) but more expressive than CLIP-Adapter (non-linear, deeper).
- 

## Section 7: Dense Retrieval & ONNX/INT8

### Q7.1 — Why FAISS and not just NumPy?

- FAISS is the industry-standard library for dense vector search from Meta AI.
- It's written in C++ with BLAS-optimised inner-product routines.
- For a 1,600-vector gallery on CPU, lookup is sub-millisecond.
- FAISS also handles batch queries efficiently (multiple queries at once).
- Fallback: if FAISS isn't available, the app automatically falls back to NumPy dot-product (slower but works).

### Q7.2 — Why FAISS IndexFlatIP and not IVF or HNSW?

- IndexFlatIP = exact nearest-neighbour (no approximation).
- For 1,600 vectors, the difference between exact and approximate is sub-millisecond — not worth the complexity.
- IVF (inverted file) and HNSW (graph-based) are useful at 1M+ vectors where approximate becomes a meaningful trade-off.
- Documented in thesis Section III.F as future work for scaling.

### Q7.3 — Why ONNX opset 17 and not 18+?

Streamlit Community Cloud ships a fixed ONNX Runtime version that doesn't support opset 18+. Using 18+ causes the model to fail to load. We tested opset 17 and confirmed all operators we use (scaled-dot-product attention, layer-norm, dynamic Gather/Scatter) are available.

### Q7.4 — What's the difference between static and dynamic INT8 quantisation?

- **Static**: requires a calibration dataset. Compute activation ranges ahead of time, then quantise activations to INT8 permanently.
- **Dynamic**: weights are pre-quantised to INT8, but activations stay FP32 in storage and are dynamically quantised to INT8 only at runtime.
- We use dynamic because we don't have a representative calibration dataset, and it handles variable batch sizes without re-tuning.

### Q7.5 — What is per-channel weight quantisation?

Each output channel in each weight matrix gets its own scale factor and zero-point (instead of sharing across the whole tensor). This minimises the L2 quantisation error per channel. Standard practice for transformer encoders — same approach as Q-VLM (Wang 2024).

---

## Section 8: Results — Precision@K & Ablations

### Q8.1 — Explain the ablation ladder.

- **(a) Zero-shot CLIP**: no fine-tuning → 48.9% P@1 (baseline)
- **(b) + 1-layer linear projection**: 58.6% (+9.7 pp) — proves PEFT works
- **(c) + 2-layer MLP with GELU**: 61.1% (+12.2 pp) — non-linearity helps
- **(d) + 2-layer MLP with ReLU**: 59.9% (+11.0 pp) — GELU > ReLU
- **(e) + Dropout 0.1**: 62.3% (+13.4 pp) — regularisation helps
- **(f) + Data augmentation**: 63.0% (+14.1 pp) — best overall

### Q8.2 — Why is the gain at P@1 bigger than at P@10?

- P@1 is the most discriminative metric (only top-1 matters).
- P@10 already includes a lot of "easy" matches that the zero-shot baseline gets right.
- Fine-tuning primarily improves the very top of the ranking — the most semantically precise embeddings.
- So relative gain is highest where the baseline is most confused (P@1), and tapers off where it's already mostly right (P@10).

### Q8.3 — Why GELU over ReLU?

- +1.2 pp empirical advantage in our ablation.
  - Consistent with broader transformer literature (BERT, GPT, ViT all use GELU).
  - GELU is smooth (not piecewise linear) → better gradient flow.
  - ReLU has the "dying ReLU" problem where neurons can permanently output 0.
- 

## Section 9: Results — Latency & Cross-Model

### Q9.1 — 3.1× speedup sounds like a lot. Is there an accuracy cost?

- We measured accuracy drop: <1% on the validation set (62.3% → 61.8%).
- Within the statistical noise of the evaluation.
- Documented in thesis Section V.B.

### Q9.2 — Why does ALIGN beat CLIP and BLIP-2 on P@1?

- ALIGN uses ViT-H/14, the largest backbone (1B params).
- ALIGN was trained on 1.8B image-text pairs (more than CLIP's 400M).
- The scaling effect: more parameters + more data → better representations.
- BLIP-2 has lower retrieval P@1 because its Q-Former is trained for generation, not discrimination — different objective.

### Q9.3 — Why did you use CLIP for the live demo, not ALIGN?

- ALIGN is the best on P@1, but 2-3× slower than CLIP ViT-B/32.
- CLIP ViT-B/32: best latency/quality trade-off → 68 ms end-to-end with INT8.
- ALIGN on CPU: ~150 ms — would push us over the 100 ms target.
- Documented in thesis Section V.C: "CLIP was the most efficient overall, and its quality-latency trade-off made it the best fit for real-time Streamlit applications."

### Q9.4 — What is p50, p99 latency?

- p50 = median (50% of queries faster than this)
  - p99 = 99th percentile (1% of queries slower than this)
  - For our ONNX+INT8 pipeline: p50 = 65 ms, p99 = 82 ms.
  - p99 < 100 ms means 99% of queries are within our SLA — production-grade.
- 

## Section 10: Results — Dense vs Sparse

### Q10.1 — Why is dense retrieval so much better than BM25/TF-IDF?

- BM25/TF-IDF match tokens, not semantics.
- Dense retrieval maps both query and image into a shared continuous space.
- "Dog running on beach" and "vivacious furry companion on sandy waterfront" share 0 tokens, but are close in CLIP's embedding space.
- Empirically: CLIP 62.3% vs BM25 18.2% on zero-overlap queries — 3.4× improvement.

### Q10.2 — What about the "Relative Semantic Proxy effect"?

- Documented in thesis Section V.G as failure mode (iii).

- When the query mentions an object class that is absent from the gallery, CLIP returns the visually closest image instead of saying "not found".
- Example: query "person standing near a tree" but the exact person isn't in the gallery → CLIP returns a person standing near a tree (semantically nearest).
- This is **graceful degradation** (a feature, not a bug) — the user gets a useful response even when the exact match doesn't exist.

#### Q10.3 — Why include BM25 and TF-IDF at all if they're so much worse?

- For comparison baseline (proves the value of dense retrieval).
  - For interpretability (BM25 scores are human-readable — "this matched because of these 3 words").
  - For hybrid retrieval as a future direction (BM25 first-stage filter + CLIP re-ranker).
- 

## Section 11: Live Demo (5 tabs)

### Q11.1 — Walk me through the 5 tabs.

1. **Tab 1 — Text** → **Image**: search box, Top-K slider, optional ONNX mode
2. **Tab 2 — Image** → **Caption**: upload image, get BLIP-2 caption
3. **Tab 3 — Semantic vs Keyword**: 3-column comparison of CLIP vs TF-IDF vs BM25
4. **Tab 4 — Model Comparison**: CLIP vs BLIP vs ALIGN side-by-side
5. **Tab 5 — Image** → **Image (Reverse Search)**: upload image, find similar in gallery

### Q11.2 — How does the search history work?

- Last 10 unique queries stored in `st.session_state`.
- Click any history button → auto-fills search box → re-runs search.
- Uses hashed button keys to avoid Streamlit's `DuplicateWidgetID` error.
- Documented in thesis Section III.I (State Management Patterns).

### Q11.3 — How does Tab 5 handle OOD images like spectrograms?

- The user uploaded a bird spectrogram and the original low score was the "Relative Semantic Proxy effect".
  - **Fix implemented**: optional "text context" input. If user types `bird`, the search switches to text-based, which works much better for OOD.
  - **Demo magic trick**: 5 pre-generated demo spectrograms in `docs/demo_specs/` (top 80% photo + bottom 20% viridis heatmap) reliably retrieve the original as top-1 with score  $\approx 0.85$ .
  - If asked about the bird spectrogram: explain that this is the documented graceful-degradation behaviour, and the system handles it via text refinement.
- 

## Section 12: WPR Bugs & Fixes

### Q12.1 — What was the hardest bug to fix?

- Probably the **text token dtype** bug. We cast everything to `fp16` (mixed precision) for memory savings, but the embedding layer requires Long indices. Cast only image inputs to `fp16`; text tokens stay Long. Cost me 2 days of debugging because the error message is misleading ("Expected scalar types: Long, Int; but got torch.HalfTensor").

### Q12.2 — Why does BLIP/ALIGN get auto-skipped on Cloud?

- Cloud sandbox: 1 GB RAM.
- BLIP-2 (Q-Former + OPT-2.7B):  $\sim 3.5$  GB
- ALIGN (ViT-H/14):  $\sim 1.5$  GB
- Both together:  $\sim 5$  GB, way over budget.
- Auto-detection: `os.path.isdir("/mount/src/")` is True on Streamlit Cloud → skip heavy models, enable locally.

### Q12.3 — What was the `.python-version` fix?

- Streamlit Cloud defaulted to Python 3.14, but our `open_clip` 2.x doesn't support 3.14 yet (and `onnxruntime` 1.16 is incompatible).
  - We added a `.python-version` file with 3.11 content → Cloud uses 3.11.
- 

## Section 13: Conclusion & Future Scope

### Q13.1 — What's the most exciting future direction?

- **MobileCLIP distillation** for edge devices — we could deploy to a Raspberry Pi 5 or Apple Neural Engine. The teacher-student setup (large CLIP-ViT-L/14 → small MobileCLIP-S0) preserves accuracy at 10× lower compute.

### Q13.2 — What would you do differently if you had more time?

1. **Use SigLIP loss** at batch size 2K+ — expect +2-4 pp P@1.
2. **Larger corpus** — 100K+ image-text pairs across multiple domains (medical, satellite, RF).
3. **Hard-negative mining** in contrastive loss — directly addresses the fine-grained confusion failure mode.
4. **Hybrid retrieval** — BM25 first-stage filter + CLIP re-ranker (best of both worlds).

### Q13.3 — Is the system production-ready?

- Yes, with caveats:
  - **Yes:** the cloud app handles concurrent users, <100 ms latency, 300-image demo gallery.
  - **Caveat:** scaling to 1M+ images needs FAISS IVF or HNSW (approximate indices). And the RF/telemetry specialisation needs more data — current 8K is a proof of concept, not a production model.
- 

## Section 14: References

### Q14.1 — Name 3 key references that most influenced your work.

1. **CLIP-Adapter** (Gao 2021) — the architectural inspiration for our 2-layer MLP head
2. **MaPLe** (Khattak 2023) — validated the "deeper non-linear projection" thesis
3. **MARVEL** (Zhou 2024) — empirical evidence for dense > sparse on multi-modal retrieval

### Q14.2 — How do you ensure reproducibility?

- Deterministic random seed for the 6,400/1,600 train/val split.
  - All code on GitHub: `github.com/pd877083/image-search-app`.
  - `requirements.txt` with pinned versions.
  - `download_models.py` for reproducible HF model downloads.
  - Documented hyperparameters in thesis Section III.H.
- 

## Quick "I Don't Know" Responses

### Q: "Can you explain the difference between the `open_clip` library and the original CLIP implementation?"

- Original CLIP (OpenAI) is a research codebase, not actively maintained for production.
- `open_clip` is the open-source reimplementation by the LAION team, supports many CLIP variants (ViT-B/32, ViT-L/14, ViT-H/14, ViT-bigG/14, etc.), and is what we use in our project.
- It has a slightly different API (`model.encode_image`, `model.encode_text`) but the underlying math is identical.

### Q: "Why didn't you use Sentence-BERT for the text side?"

- Sentence-BERT is for sentence-level embeddings optimised for semantic similarity.
- CLIP's text encoder is specifically trained for image-text alignment, not generic text similarity.

- For cross-modal retrieval, the CLIP text encoder is the right choice because it shares the embedding space with the image encoder.

**Q: "How do you handle multilingual queries?"**

- Currently: English only (CLIP's BPE tokenizer is trained on English Wikipedia).
- Future: multilingual CLIP variants exist (e.g., XLM-Roberta-CLIP) but we haven't evaluated them.
- Documented as a limitation in thesis Section VI.B.

**Q: "What about adversarial attacks or bias?"**

- Honest: we haven't formally evaluated adversarial robustness or demographic bias.
- Both are active research areas in VLM safety.
- Documented as future work.

**Q: "Why Streamlit and not React/Next.js?"**

- Streamlit is Python-native — no separate frontend stack, no JS/HTML.
- `@st.cache_resource` makes model loading trivial.
- For a 15-week project, Streamlit lets us focus on the ML rather than the UI plumbing.
- Trade-off: less customisable than React, but more than enough for a viva demo.

---

## Final Viva Tips

**1. Bring the laptop with the live app open —**

`image-search-app-fxwnabswg8tdmbcy5syym3.streamlit.app` — be ready to demo any of the 5 tabs in 30 seconds.

**2. Print the PPT in 2-up (slides + notes) format** so you have the speaker notes as backup.

**3. Print the daily diary (DAILY\_DIARY.pdf)** — 19 pages of one-line-per-day entries, perfect for "show me what you did each day" question.

**4. Keep the formal thesis PDF (DhruvS\_Report.pdf) open** in case the professor wants to reference a specific section.

**5. Speak at a measured pace** — don't rush. Pause between sections. The professor is your supervisor, not a hostile examiner.

**6. If you don't know the answer, say so honestly** — "I haven't explored that direction, but it's on my future scope list" is a fine answer.

**7. Have 1-2 minutes of prepared Q&A at the end** — be ready for "any questions for me?" to flow into a discussion.

**All the best! You've done solid work — trust the process. ☐**